

Unsupervised Anomaly Detection: MVTec Dataset

INTRODUCTION

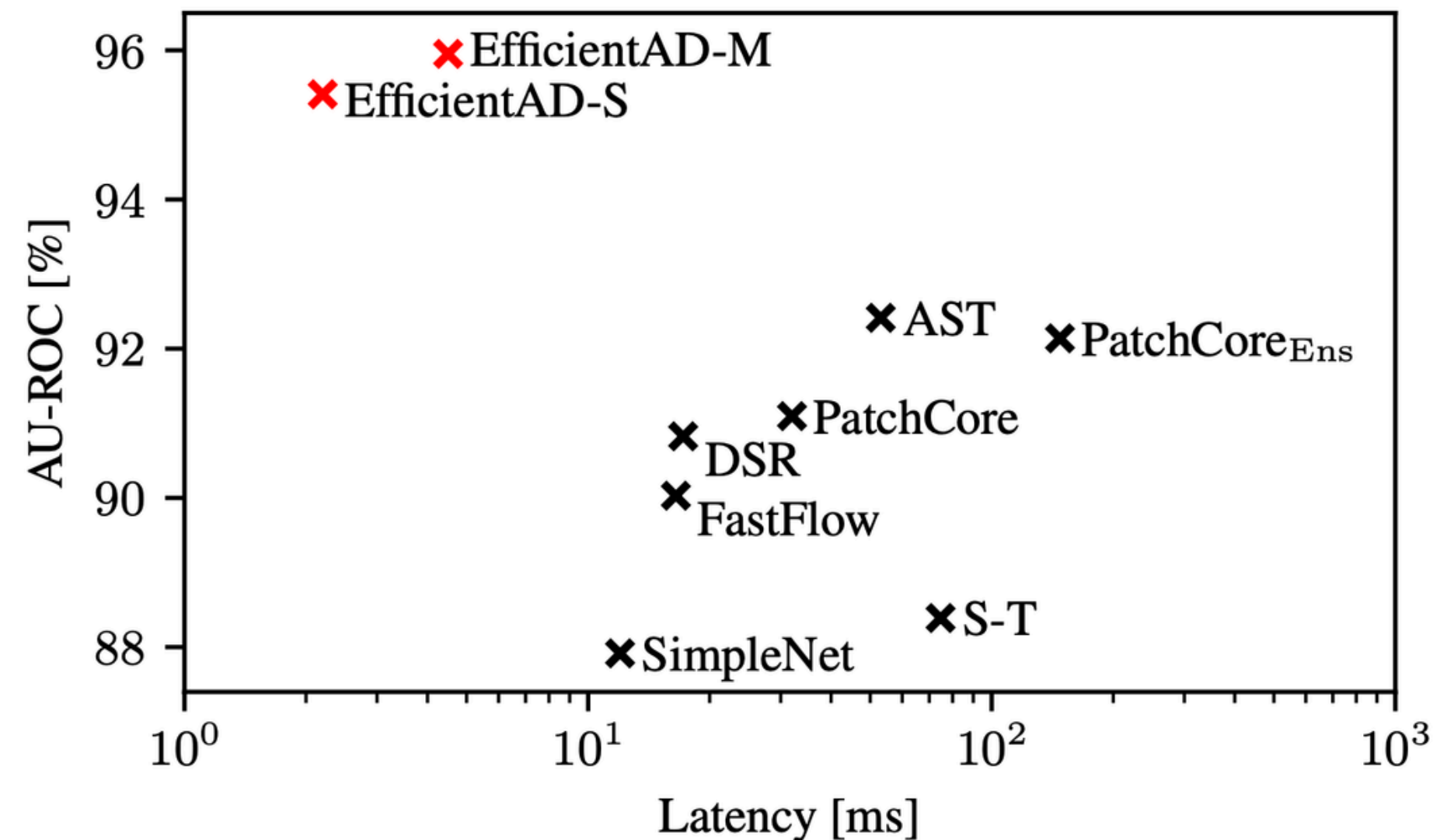
- Can a model be trained to detect 'never-seen-before' defects with high precision/recall?

- How do we create robust production-grade models for industrial settings?

Where do the bottlenecks lie?

- How do we find and evaluate the 'perfect model'?

State of the Art



- Y-Axis (Accuracy): image-level anomaly detection performance, measured by the AU-ROC. Average score across 32 distinct inspection scenarios from three major dataset collections: MVTec AD, VisA, and MVTec LOCO. Higher is better.
- X-Axis (Speed): inference latency, on an NVIDIA RTX A6000 GPU. Lower is better.

Implementation

PatchCore

- WideResNet50 (frozen, ImageNet pretrained)
- Forward hooks on layer2 + layer3
- Greedy coreset subsampling
- FAISS flat index (memory bank)

EfficientAD

- Pretrained teacher PDN (frozen)
- Student PDN (2× channels, trainable)
- Convolutional autoencoder with dropout decoder
- Three losses: ST, AE, ST-AE bridge

	PatchCore	EfficientAD
MEMORY	HIGH	LOW
INFERENCE	HIGH	LOW
TRAINING	LOW	HIGH

TABLE: Comparison of 2 SOTA models

RESULTS

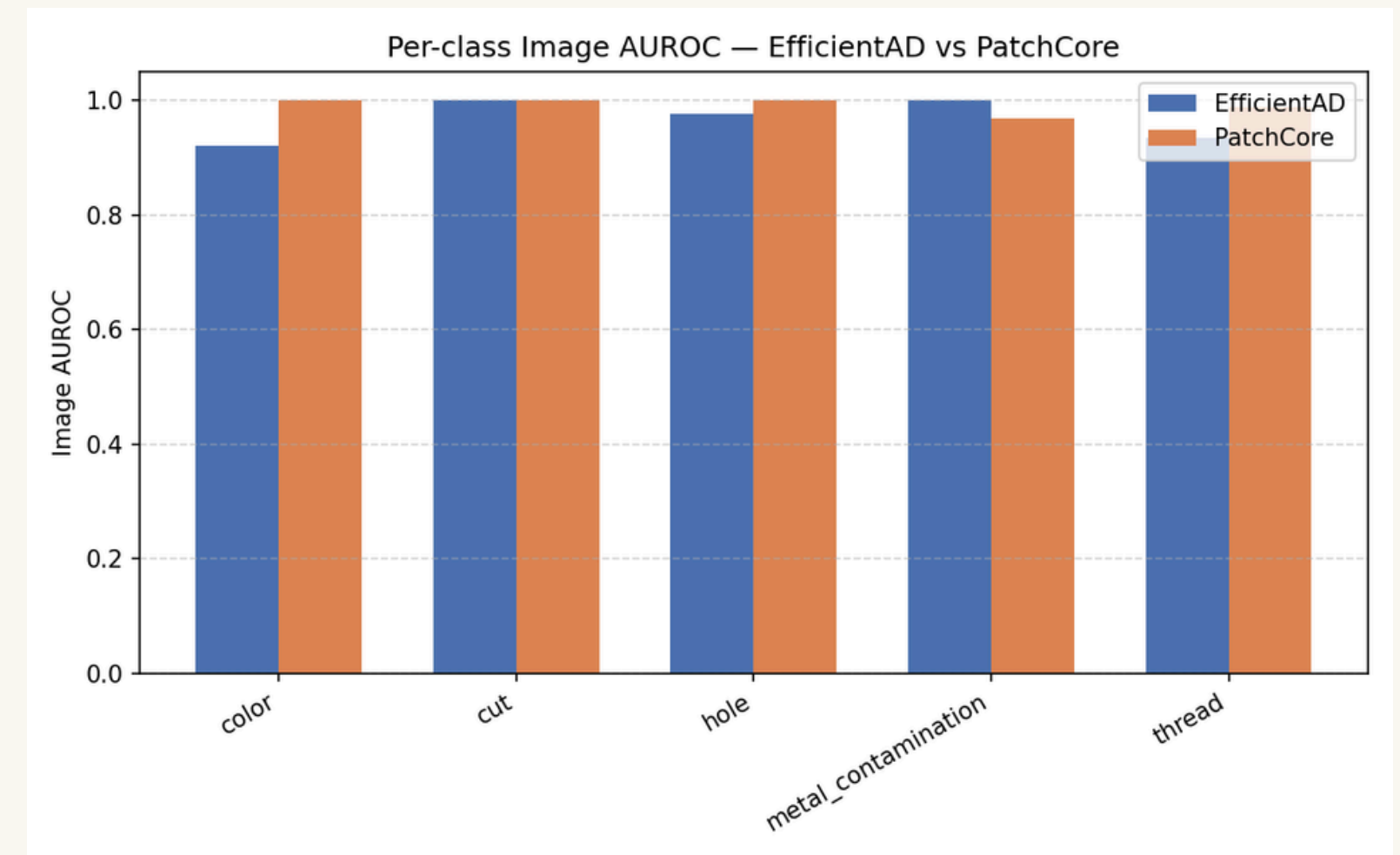
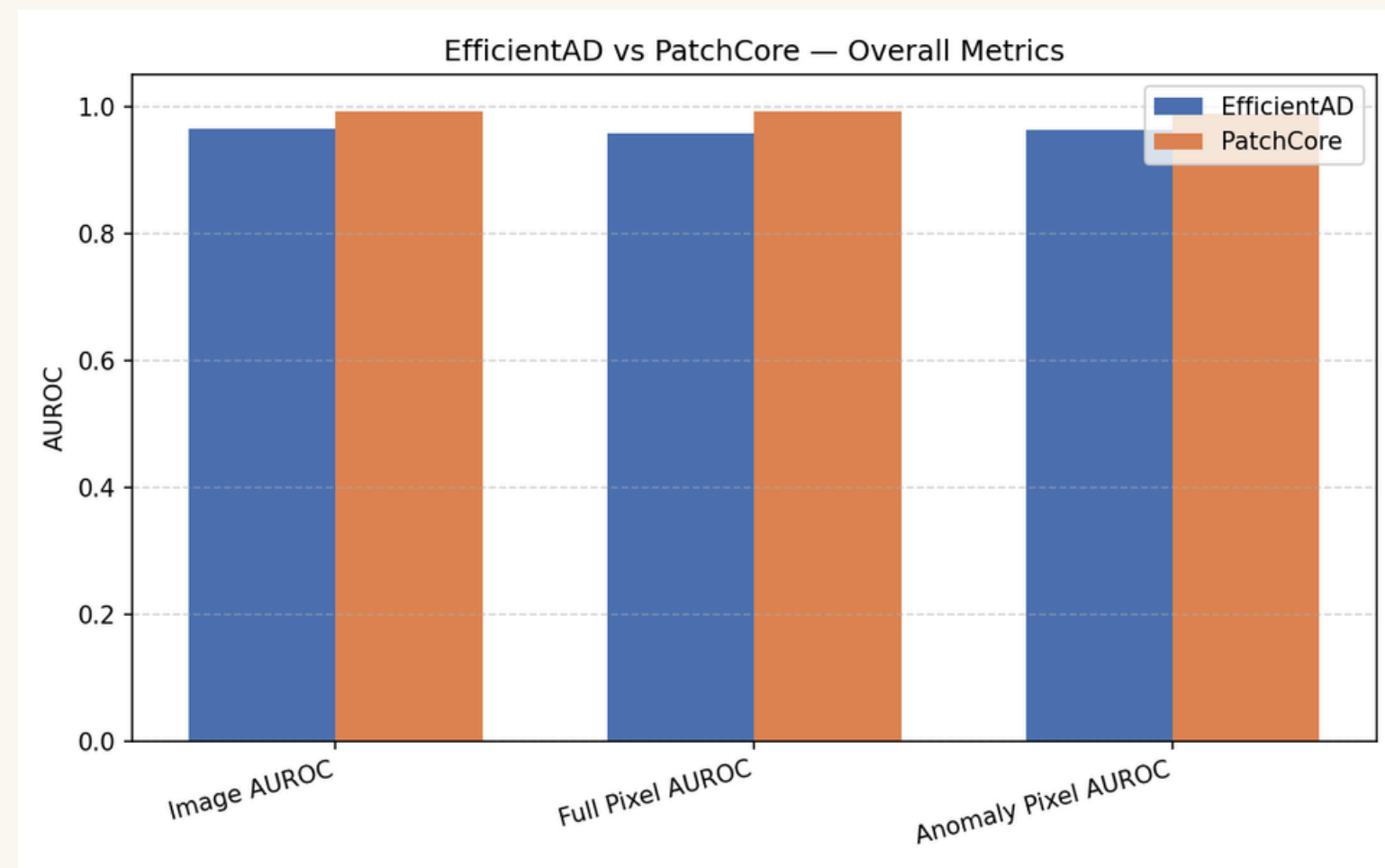


Fig: EfficientAD vs PatchCor AUROC metrics

RESULTS

<u>Model</u>	<u>Mean (ms)</u>	<u>Min (ms)</u>	<u>Max (ms)</u>
EfficientAD (CUDA)	14.6	14.53	15.0
PatchCore (CUDA)	380.84	380.30	381.71
EfficientAD (CPU)	492.62	474.86	495.8
PatchCore (CPU)	614.25	586.37	634.12

Table: Inference latency for test set

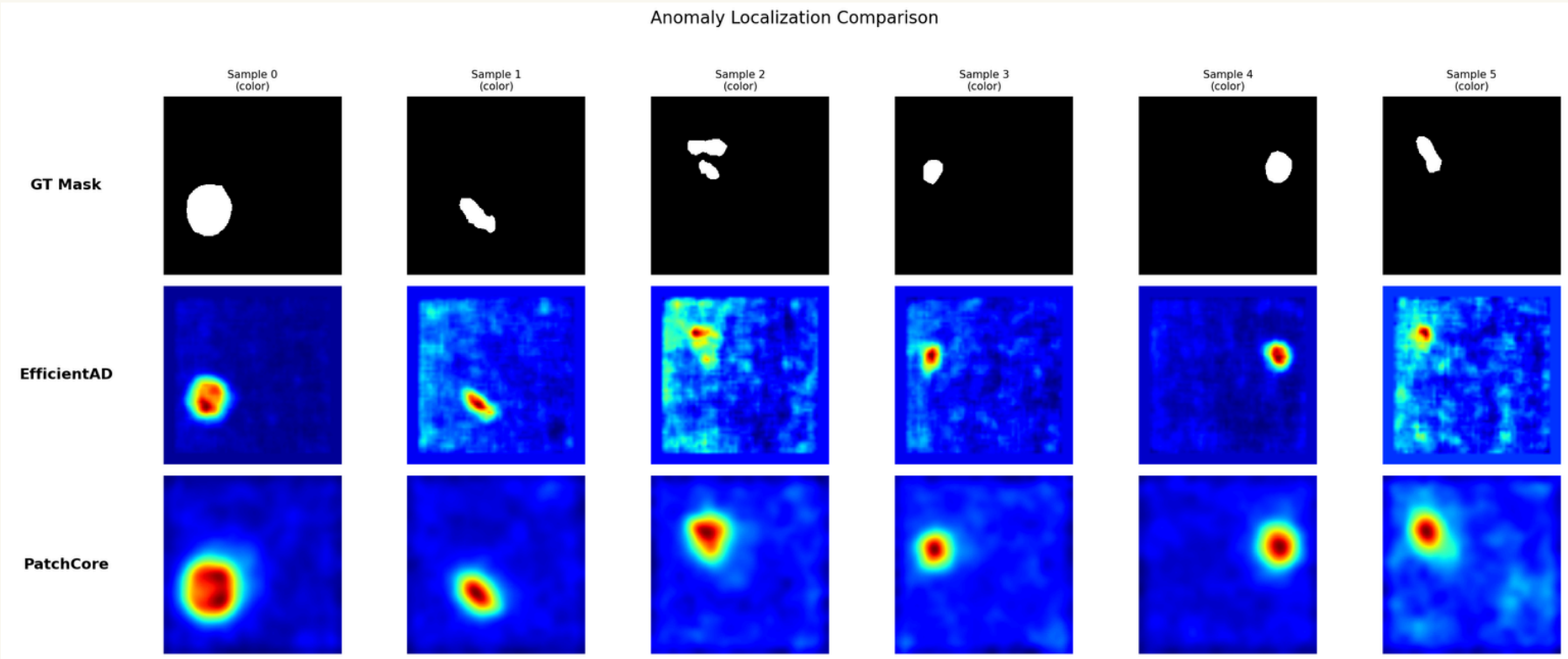


Fig: Anomaly detection maps per model